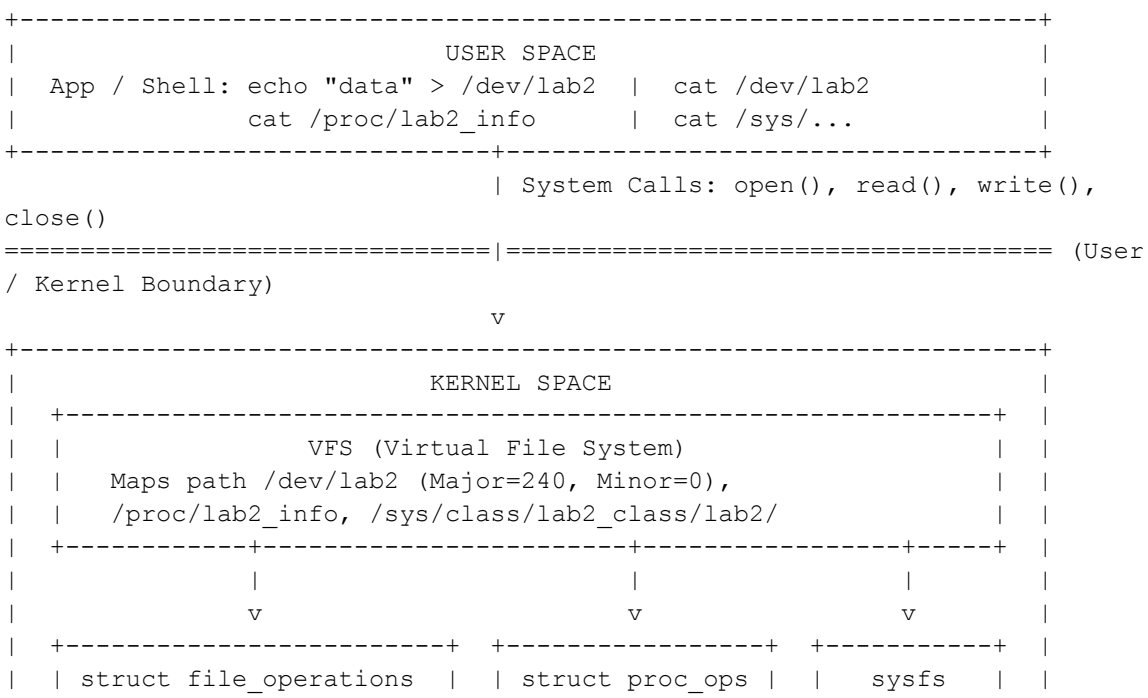
EMBEDDED LINUX SYSTEMS - LABORATORY REPORT
LABORATORY EXERCISE 02 (LAB-02): WRITING DEVICE DRIVERS & FILE SYSTEM MANAGEMENT

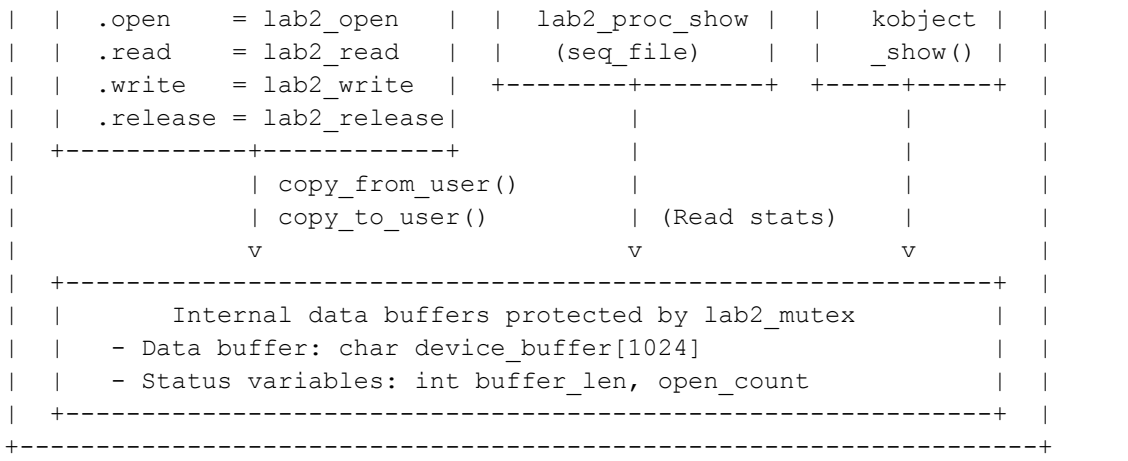
1. Student Information and Practical Environment

Full Name	Tran Gia Phuc
Student ID	SE204417
Course / Lab Item	Embedded Linux Systems — LAB-02
Date of Execution	October 17, 2026
Host Operating System	Ubuntu Linux x86_64
Target Architecture & Board	ARMv7-A – Cortex-A9 (Emulated via QEMU vexpress-a9, 2 CPU cores, 512MB RAM)
Linux Kernel Version	Linux 5.15.0 (Vermagic: 5.15.0 SMP mod_unload ARMv7 p2v8)
Cross-Compiler Toolchain	arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) version 15.2.0

2. Driver Architecture Diagrams and sysfs Structure

2.1. Data Flow Diagram from User Space to Kernel Space





2.2. Directory Tree Structure of /sys/class/lab2_class/lab2/

```
/sys/class/lab2_class/lab2/
├── buffer_len      (Permission 0444, Read-Only: outputs number of bytes
currently in device_buffer)
├── open_count      (Permission 0444, Read-Only: outputs total number of
times the device was opened)
├── last_data       (Permission 0444, Read-Only: outputs the last string
written to the buffer)
├── dev             (Kernel default file: contains major:minor pair
"240:0")
├── power/          (Default power management directory for the kobject)
├── subsystem -> ../../../../class/lab2_class
└── uevent          (Device event notification file for udev/mdev manager)
```

3. Practical Results and Logs (Parts A, B, C, D)

3.1. Part A – Building a Kernel Module for Character Device (/dev/lab2)

Cross-compiling lab2_driver.ko for ARMv7-A architecture using KBuild (with KCFLAGS=-march=armv7-a), verifying 32-bit ARM ELF binary format and kernel vermagic string before integrating into the rootfs.

```
/home/phuc/embedded_lab2/driver
x86_64
arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0

KDIR := $(HOME)/embedded_lab1/kernel/linux-5.15
ARCH := arm
CROSS_COMPILE := arm-linux-gnueabihf-

obj-m += lab2_driver.o

all:
	$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE) modules

clean:
	$(MAKE) -C $(KDIR) M=$(PWD) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE) clean

phuc@phuc-Inspiron-14-5430:~/embedded_lab2/driver$
```

Figure IMG-01: Verifying Ubuntu x86_64 host environment, ARM GCC 15.2.0 cross-compiler version, and Makefile configuration.

```
-rw-rw-r-- 1 phuc phuc 133K Oct 17 11:42 lab2_driver.ko
lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=ac9
8e4e7d08b359168688a32cfd5a3985bcf625d, with debug_info, not stripped
phuc@phuc-Inspiron-14-5430:~/embedded_lab2/driver$
```

Figure IMG-02: Checking size and ARM 32-bit EABI5 binary format of lab2_driver.ko after compilation.

```
filename:       /home/phuc/embedded_lab2/driver/lab2_driver.ko
version:        1.0
description:    LAB-02: Simple Character Device Driver
author:         Embedded Linux Lab
license:        GPL
srcversion:     4E89036A2D2BA07B7DE24B7
depends:
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
phuc@phuc-Inspiron-14-5430:~/embedded_lab2/driver$
```

Figure IMG-03: Verifying module metadata (modinfo) and vermagic string compatible with Linux Kernel 5.15.0.

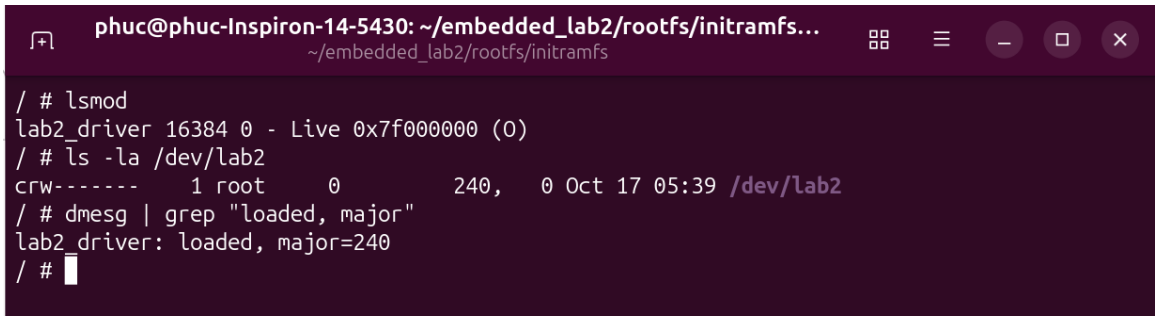


Figure IMG-04: Loading module into kernel on QEMU, checking active modules (lsmod) and device node /dev/lab2.

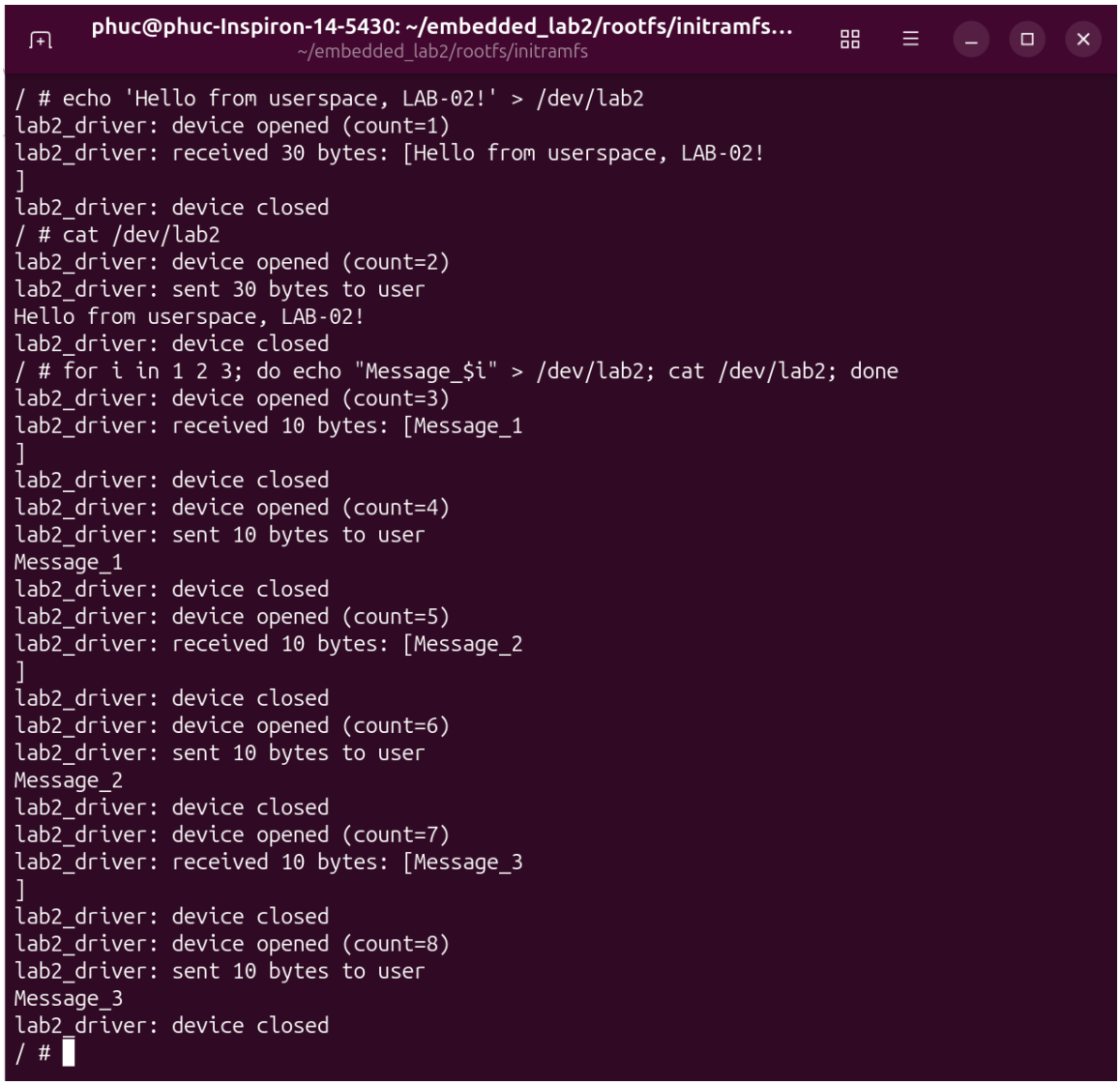


Figure IMG-05: Testing write and read operations multiple times through the character device /dev/lab2.



Figure IMG-06: Kernel log (dmesg) recording open, write, read, and release callback executions.

3.2. Part B – Communication via procfs and sysfs Virtual File Systems

Adding driver statistics reporting via /proc/lab2_info (using seq_file API) and individual attributes under /sys/class/lab2_class/lab2/.

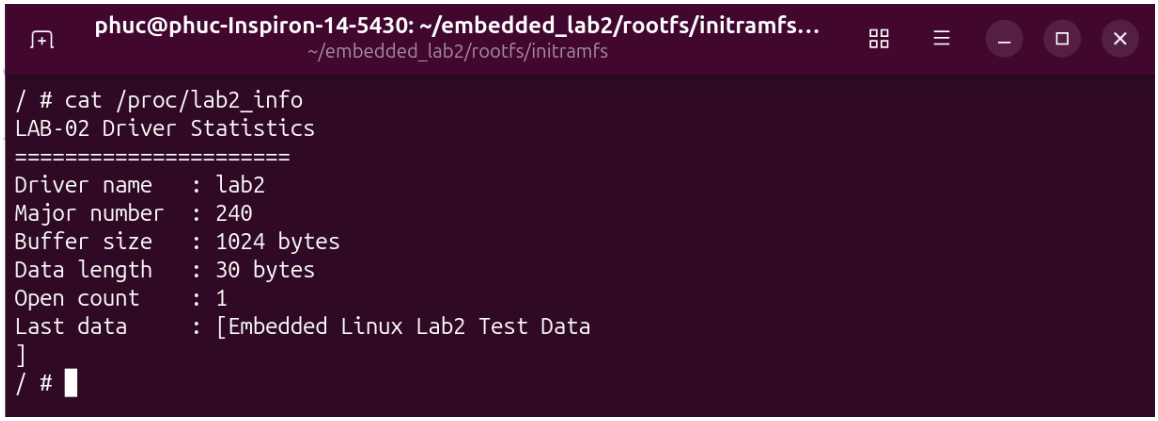


Figure IMG-07: Driver status statistics report when reading from /proc/lab2_info.

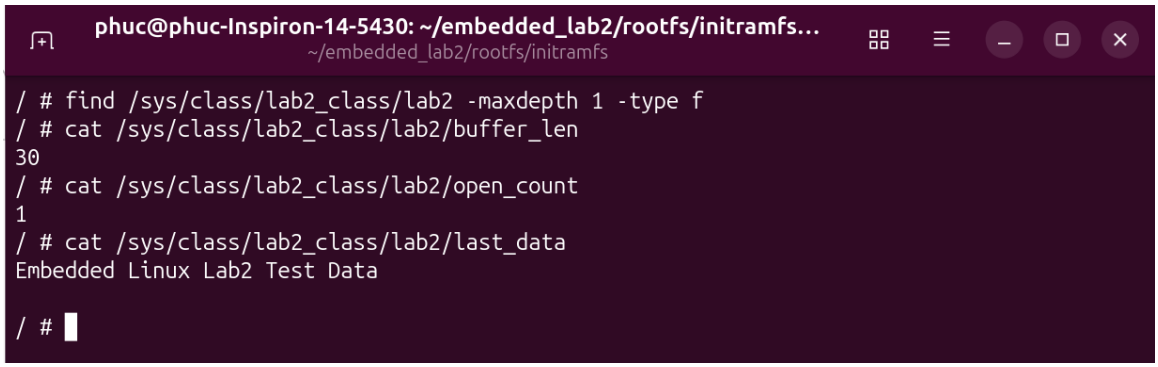


Figure IMG-08: List of attribute files in /sys/class/lab2_class/lab2/ and values read from buffer_len, open_count, last_data.

3.3. Part C – NAND Flash Emulation with nandsim, MTD, and JFFS2 File System

Experimental Note: Part C is executed directly on the Ubuntu host with root privileges (sudo). When inspecting /proc/mtd, mtd0 on the host is the NOR flash containing the motherboard BIOS; hence the emulated NAND partition created by nandsim ('NAND simulator partition 0', 32MB capacity, 16KiB eraseblock = 0x4000) resides at /dev/mtd1 and /dev/mtdblock1.

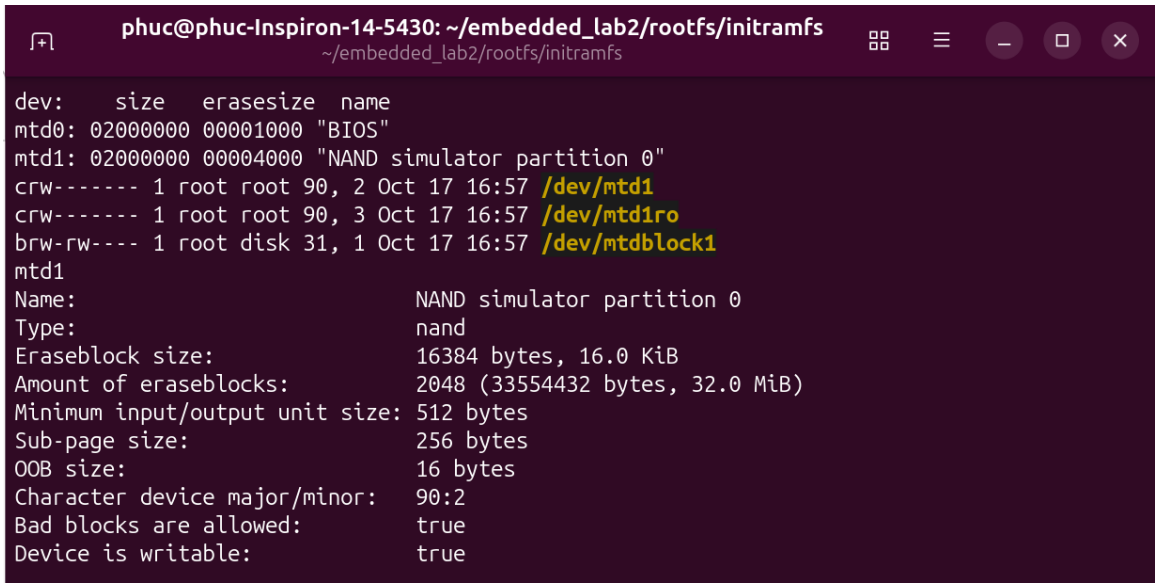


Figure IMG-09: Activating nandsim module on Ubuntu host, inspecting /proc/mtd list and technical specs of /dev/mtd1 using mtdinfo.

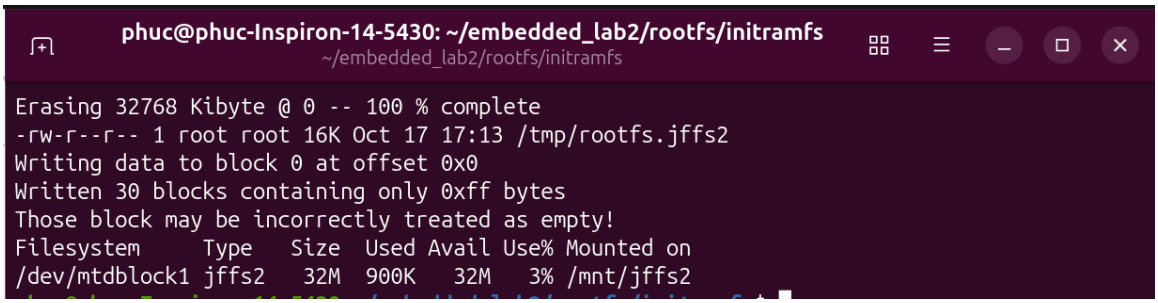


Figure IMG-10: Erasing /dev/mtd1, creating JFFS2 image with eraseblock size 0x4000, writing via nandwrite and mounting to /mnt/jffs2.

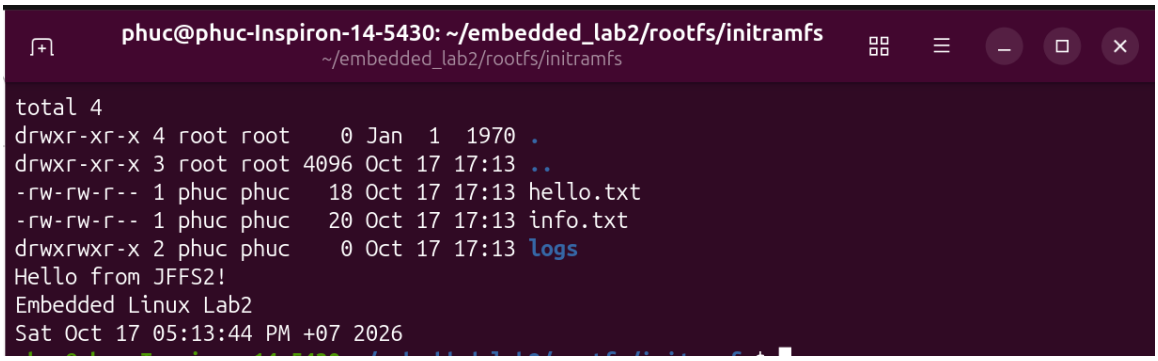


Figure IMG-11: Checking file list and reading sample data content on the JFFS2 partition at /mnt/jffs2.

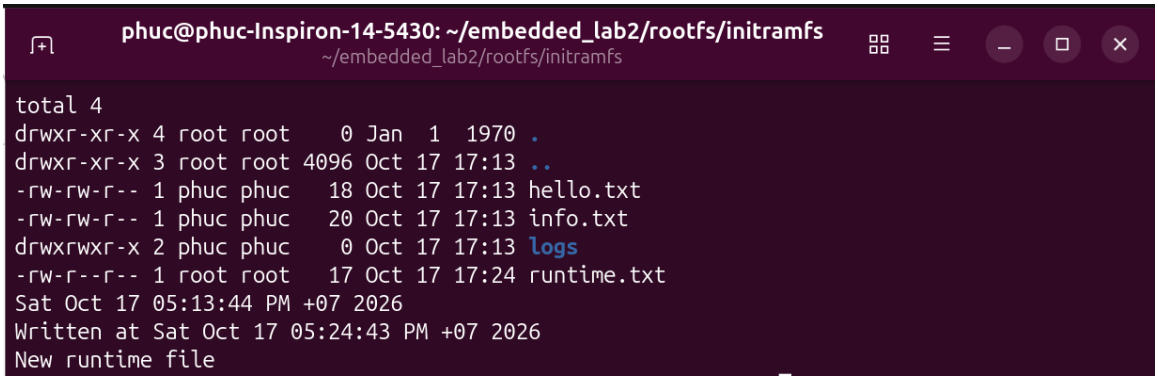


Figure IMG-12: Verifying JFFS2 persistence: writing new runtime.txt file, unmounting then remounting to ensure data is preserved.

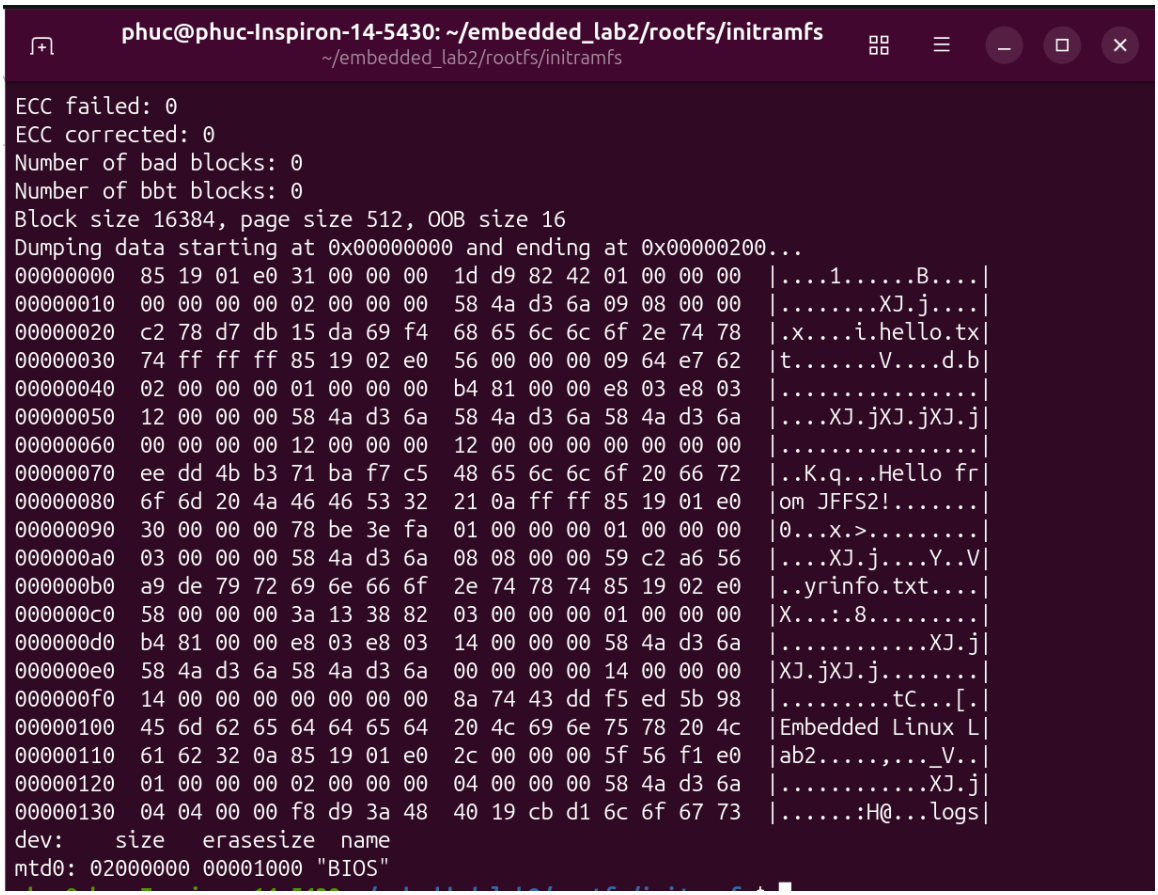


Figure IMG-13: Extracting first 512 raw bytes from /dev/mtd1 using nanddump and observing JFFS2 node structure (magic signature 85 19) via hexdump -C.

3.4. Part D – Integrating into BusyBox Init for Automatic Module Loading at Boot

Updating /etc/inittab and system initialization script /etc/init.d/rcS to automatically mount virtual file systems, load lab2_driver.ko and create device node /dev/lab2 upon system startup.

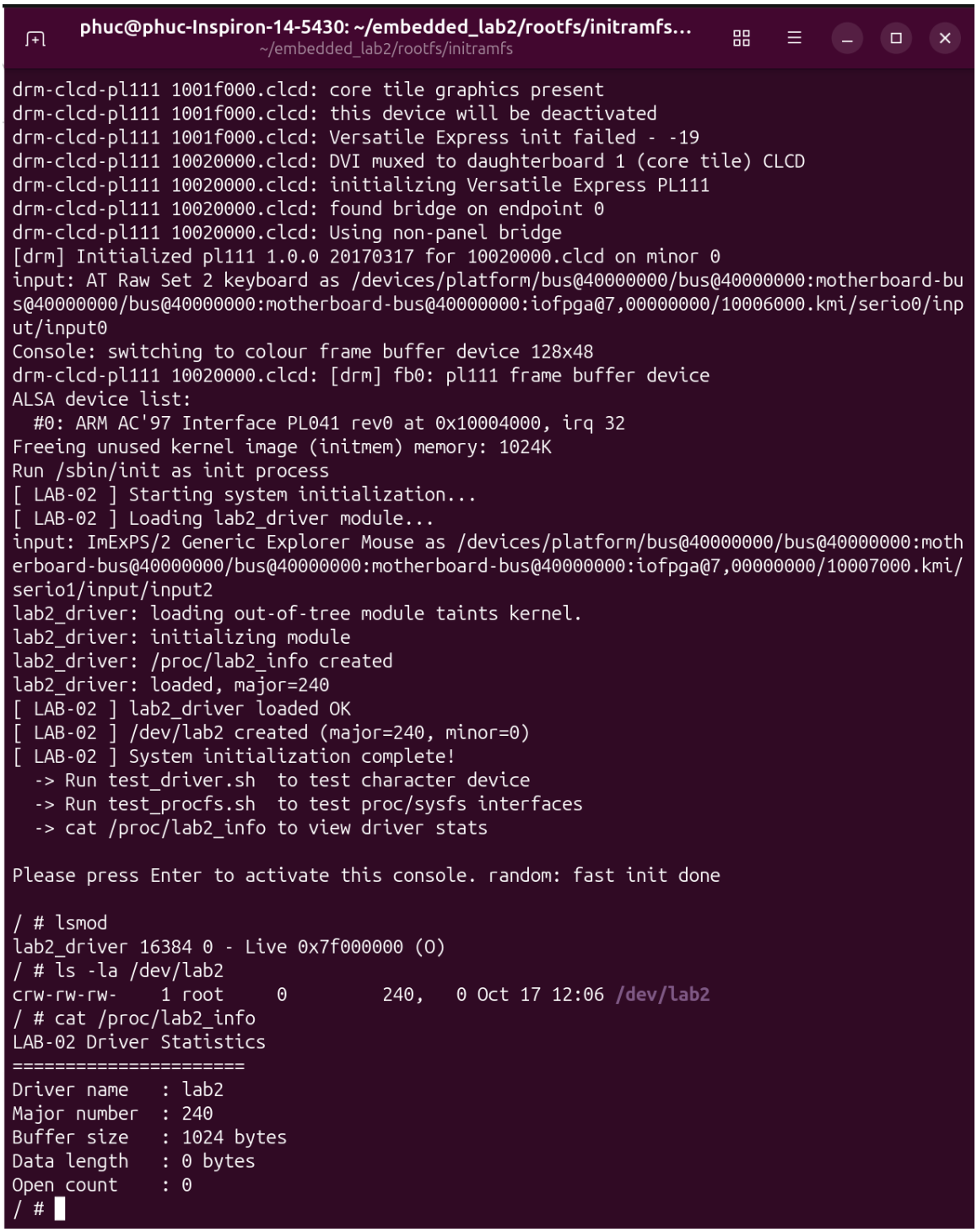


Figure IMG-14: System automatically loads lab2_driver module and initializes /dev/lab2 during QEMU boot without manual insmod commands.

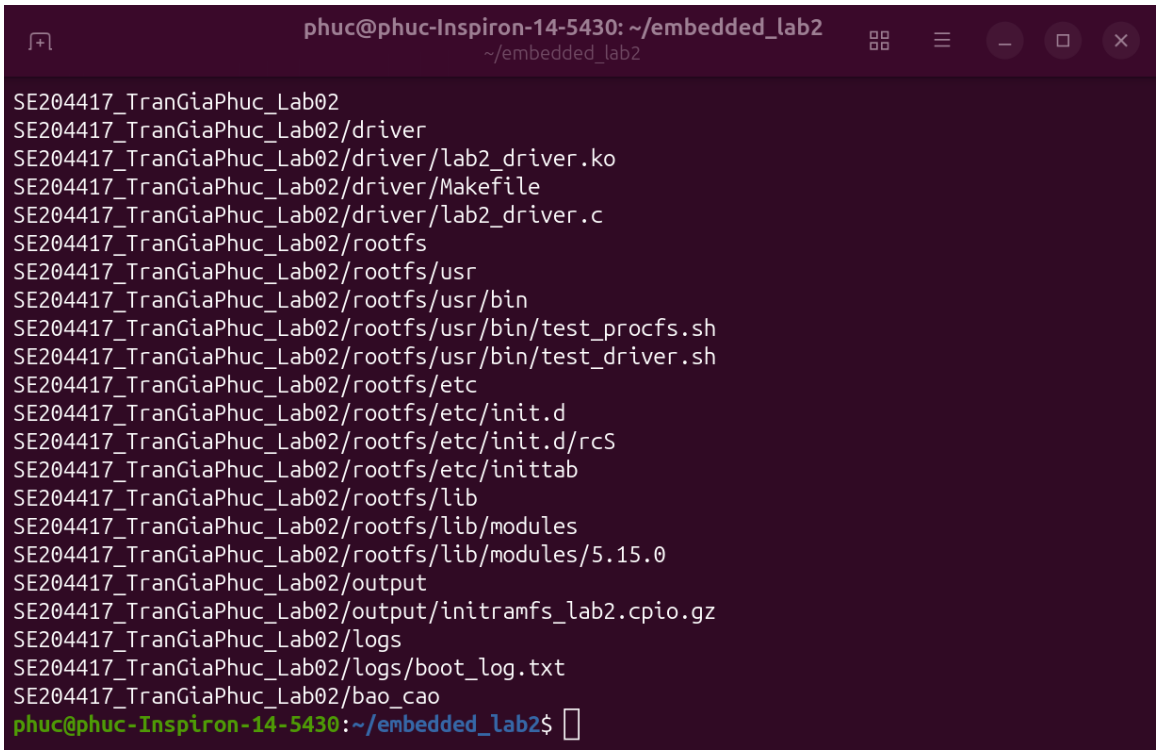


Figure IMG-15: Verifying submission directory tree SE204417_TranGiaPhuc_Lab02 according to requirements.

4. Filesystem Comparison and IoT Design Recommendations

4.1. Comparison Table of Common Filesystems in Embedded Systems

Evaluation Criteria	ext4	JFFS2	cramfs	SquashFS	UBIFS
1. Read/Write Support	Read and Write	Read and Write	Read-Only	Read-Only	Read and Write
2. Wear Leveling	None built-in (relies on hardware controller for SD, eMMC, SSD)	Built-in at filesystem level (sequential log-structured design)	Not needed (read-only, no erase/write at runtime)	Not needed (read-only)	Built-in (via UBI management layer across the entire chip)
3. Data Compression	No default compression	Supported (zlib, lzo, rtime upon writing nodes)	Supported (zlib, size limit < 256MB)	High compression (supports xz, lzo, lz4, zstd)	High-speed compression (lzo, zlib, zstd)
4. Boot and	Fast	Slower as	Very fast	Very fast	Fast (thanks

Mount Time		storage size increases (must scan all nodes during mount)			to index tree stored directly on flash)
5. Suitable Storage Devices	Block devices (eMMC, SD card, SSD/HDD)	Raw NOR/NAND Flash small capacity (< 32MB – 64MB) via MTD	ROM or small NOR Flash for static firmware	Static rootfs needing protection against file corruption	Raw NAND Flash large capacity (> 64MB)

4.2. Recommendations for an IoT Device with 32MB NAND Flash Requiring OTA Updates

For an IoT device using a raw 32MB NAND flash chip and requiring secure Over-The-Air (OTA) firmware updates, the optimal partition scheme combines SquashFS and JFFS2:

• Bootloader Partition (mtd0 - 1MB): Stores U-Boot and environment variables to select the boot bank.

• Dual Bank OS Partitions A and B (mtd1: 12MB, mtd2: 12MB) using SquashFS: Utilizing SquashFS read-only filesystem compresses the Linux kernel and rootfs into a tiny footprint, enabling fast boot times and absolute protection against power outage corruption. During OTA updates, the active bank downloads the new image into the inactive bank, verifies SHA-256 checksums, and switches the bootloader flag. If an update fails, it automatically rolls back.

• Data & Log Partition (mtd3 - remaining 7MB) using JFFS2: Dedicated to runtime data (Wi-Fi config, security certificates, log files). With a small 7MB footprint on raw NAND, JFFS2 excels due to built-in wear leveling, bad block management, compression, and power-fail data preservation without incurring slow mount penalties.

5. Analysis Questions (Section VI)

Question 1: Why cannot memcpy() be used to copy data between kernel buffer and user buffer in lab2_read() / lab2_write()?

• Separation of virtual address spaces: In 32-bit Linux on ARMv7, the 4GB virtual address space is strictly divided into User Space (0x00000000 to 0xBFFFFFFF) and

Kernel Space (0xc0000000 to 0xFFFFFFFF). User processes lack direct access to kernel memory.
• Protection mechanism of copy_to_user() and copy_from_user(): Before copying, these functions invoke access_ok() to verify whether user pointers reside within valid process address bounds (preventing malicious pointers targeting kernel memory). Furthermore, user memory might not be physically allocated (demand paging) or might be swapped out. If a Page Fault occurs during copying, the kernel Exception Table catches it, pages in the memory or aborts safely, returning -EFAULT to the driver.
• Consequences of using memcpy(): memcpy() performs raw copying without permission checks or fault handlers. Passing an invalid or unmapped pointer causes an unhandled memory access fault in Privilege Mode, triggering Kernel Oops or Kernel Panic and crashing the system.

Question 2: Why is DEFINE_MUTEX(lab2_mutex) required when multiple processes open /dev/lab2? Describe a specific race condition if mutex is absent.

• Need for Mutex: device_buffer[1024], buffer_len, and open_count are global variables shared across Kernel Space for all processes opening /dev/lab2. On symmetric multiprocessing systems (QEMU -smp 2) with a preemptive kernel, multiple user processes can invoke system calls simultaneously.
• Race condition scenario without mutex: Suppose Process A calls write() to write 'AAAAAAAAAA' (10 bytes) and Process B calls write() to write 'BBBB' (4 bytes) simultaneously. Process A writes 5 bytes ('AAAAA') into device_buffer and is preempted. Process B overwrites the first 4 bytes with 'BBBB', sets buffer_len = 4 and null-terminator at index 4. Process A resumes, writes the remaining 5 bytes into indices 5-9 (overwriting B's null-terminator) and sets buffer_len = 10.
• Consequences: Corrupted buffer content ('BBBBAAAAAA') and atomic reference counting failures. Using mutex_lock(&lab2_mutex) and mutex_unlock(&lab2_mutex) ensures serialized exclusive access.

Question 3: Compare procfs (/proc) and sysfs (/sys). Why does kernel documentation recommend /sys over /proc for new entries?

• Comparison: procfs (/proc) originated to expose process stats, later used for multi-line free-form text reports (e.g., /proc/lab2_info). Conversely, sysfs (/sys) aligns with the Linux Device Model (kobject, device_class, device), organizing attributes into a strict hierarchical tree where each file contains exactly a single value.
• Recommendation rationale: Free-form text in /proc lacks standardized formatting, forcing user-space parsers to be complex and brittle. /sys reflects hardware hierarchy and ties directly to device lifecycles, making individual attribute files cleaner and script-friendly.
• Application in lab: /proc/lab2_info presents a human-readable summary table for

debugging, while individual metrics like buffer_len (30), open_count (2), and last_data reside as separate files under /sys/class/lab2_class/lab2/ for automated scripts (test_procfs.sh) to read directly.

Question 4: Why must JFFS2 erase an entire erase block before writing, unlike ext4? How does JFFS2 log-structured mechanism solve this?

• NAND Flash physical characteristics: Floating-gate cells in raw NAND flash default to logic 1 (0xFF) after erasure. Program/Write operations (per page) can only pull bits from 1 to 0, not vice versa. Restoring bits to 1 requires high-voltage Erase across an entire erase block (16 KiB on /dev/mtd1). Meanwhile, ext4 runs on block devices with built-in Flash Translation Layer (FTL) controllers handling erase abstractions, or HDDs supporting sector-level overwrites.
• Log-structured mechanism: When updating files (e.g., boot.log or runtime.txt), JFFS2 does not overwrite in-place. Instead, it packages updates into a new JFFS2 node (prefixed with magic number 0x1985 -> '85 19') with a higher version number, appends it to free pages, and marks old nodes obsolete. Garbage Collection later reclaims obsolete blocks through erasure, overcoming physical NAND write constraints while performing wear leveling.

Question 5: If insmod fails in /etc/init.d/rcS, does the system continue booting? How to improve rcS to distinguish critical vs non-critical errors?

• Behavior on failure: The system continues booting into the shell prompt (/ #) because /etc/init.d/rcS runs sequentially without shell 'set -e'. When insmod fails (non-zero exit code), the if condition branches to print an error message, but rcS continues to completion. BusyBox init then proceeds to ::askfirst:-/bin/sh in /etc/inittab.
• Distinguishing error severities:
  - Non-critical errors (e.g., optional modules): Log the error to /dev/kmsg and allow boot to proceed so engineers can debug.
  - Critical errors (e.g., mounting /proc fails or core hardware drivers fail): Abort startup or enter safe recovery mode.
• Improved code snippet for /etc/init.d/rcS:

```
MODULE_PATH="/lib/modules/$(uname -r)/lab2_driver.ko"
CRITICAL_MODULE=0  # Set to 1 if module is mandatory
if [ -f "$MODULE_PATH" ]; then
    if insmod "$MODULE_PATH"; then
        echo "[ LAB-02 ] lab2_driver loaded successfully"
        [ -c /dev/lab2 ] || mknod /dev/lab2 c 240 0
        chmod 666 /dev/lab2
    else
        echo "[ LAB-02 ] ERROR: insmod failed (code $?)!" | tee /dev/kmsg
```

```
        [ "$CRITICAL_MODULE" -eq 1 ] && { echo "[ FATAL ] Halting startup!"; exit
1; }
    fi
 else
    echo "[ LAB-02 ] WARNING: $MODULE_PATH not found!" | tee /dev/kmsg
    [ "$CRITICAL_MODULE" -eq 1 ] && exit 1
 fi
```

6. Difficulties Encountered and Solutions

Difficulty 1 – Assembler instruction conflict when compiling module with GCC 15.2.0:

• Cause: Compiling lab2_driver.c with arm-linux-gnueabihf-gcc v15.2.0 on Ubuntu against Linux kernel 5.15 caused assembler errors regarding the memory barrier instruction 'isb' due to fallback to older ARM architectures.
• Solution: Passed KCFLAGS=-march=armv7-a to the make command (make KCFLAGS=-march=armv7-a) to explicitly target ARMv7-A (Cortex-A9), ensuring successful compilation and correct vermagic.

Difficulty 2 – /dev/mtd0 on Ubuntu host mapped to host motherboard BIOS NOR flash:

• Cause: Default guides use /dev/mtd0. However, checking /proc/mtd and sudo mtdinfo /dev/mtd0 on the Ubuntu host revealed that mtd0 points to the host BIOS NOR flash (4KiB eraseblock, read-only), whereas the nandsim partition resides at mtd1 (16KiB = 0x4000 eraseblock).
• Solution: Inspected /proc/mtd prior to operations and redirected flash_erase, nandwrite, mount, and nanddump commands to /dev/mtd1 and /dev/mtdblock1, ensuring host safety and JFFS2 eraseblock alignment.

Difficulty 3 – 'File exists' error on repeated test runs and bridging /init with BusyBox /sbin/init:

• Cause: Running test_driver.sh repeatedly triggered 'File exists' errors from insmod because the module was already loaded. Additionally, the initial LAB-01 /init script only invoked /bin/sh directly without parsing /etc/inittab.
• Solution: Added existence checks ([-c /dev/lab2] || mknod ...) in rcS, updated /init to exec /sbin/init, and passed rdinit=/sbin/init to QEMU to execute the correct BusyBox init startup sequence.